

Software Requirements Specification

DebtPilot – Personal Finance and Debt Repayment Planner

Version: 1.0

Document Status: Initial Release Specification

Prepared For: DebtPilot Development

Prepared By: Anvesh Peddelli

Technology: C++20, Qt 6, SQLite, CMake, Google Test

Date: July 2026

1. Introduction

1.1 Purpose

This Software Requirements Specification defines the functional and non-functional requirements for DebtPilot Version 1.0.

DebtPilot is a personal finance and debt repayment planning application that helps users understand their debts, compare repayment strategies, estimate total interest, and generate month-by-month repayment plans.

The document is intended for:

- Developers
 - Test engineers
 - UI designers
 - Project reviewers
 - Interviewers evaluating the project
 - Future contributors
-

1.2 Product Overview

DebtPilot allows users to enter their income, expenses, available repayment budget, and outstanding debts.

The application then calculates repayment plans using:

- Debt Avalanche strategy
- Debt Snowball strategy

The system shows:

- Monthly repayment schedule
- Debt repayment priority
- Estimated debt-free date
- Total interest payable
- Total amount repaid
- Comparison between repayment strategies

DebtPilot Version 1.0 will use deterministic financial calculations. Artificial intelligence will not be responsible for calculating balances, interest, payments, or repayment schedules.

1.3 Product Goals

The primary goals of DebtPilot are:

1. Help users understand their complete debt situation.
 2. Provide clear and accurate repayment plans.
 3. Compare different debt repayment strategies.
 4. Reduce the complexity of personal debt planning.
 5. Provide month-by-month visibility into repayment progress.
 6. Maintain user financial data locally.
 7. Demonstrate clean software architecture and modern C++ practices.
 8. Provide a foundation for future budgeting and AI-assisted financial explanation features.
-

1.4 Scope of Version 1.0

DebtPilot Version 1.0 will include:

- User financial profile creation
- Income and expense entry
- Debt entry and validation
- Monthly repayment-budget calculation
- Debt Avalanche strategy
- Debt Snowball strategy
- Monthly interest calculation
- Month-by-month repayment schedules
- Total interest calculation
- Debt-free date estimation
- Strategy comparison
- Local SQLite persistence
- Desktop graphical user interface
- Sample financial profile support
- Export of repayment schedules to CSV

- Automated unit and integration tests
-

1.5 Out of Scope for Version 1.0

The following features will not be included in Version 1.0:

- Direct bank account integration
 - Credit-card account synchronization
 - Automatic transaction imports
 - Credit-score calculation
 - Credit-score guarantees
 - Investment recommendations
 - Loan approval recommendations
 - Tax planning
 - Insurance planning
 - Mobile application
 - Cloud synchronization
 - Multi-user support
 - Real-time interest-rate updates
 - AI-generated repayment calculations
 - Payment execution
 - Automatic bill payments
 - Financial institution integrations
 - Foreign currency support
 - Cryptocurrency tracking
-

1.6 Definitions and Terminology

Debt

An amount owed by the user to a lender or financial institution.

Principal

The outstanding amount on which interest may be calculated.

Annual Percentage Rate

The yearly interest rate applied to a debt.

Minimum Payment

The minimum amount the user must pay toward a debt each month.

Extra Repayment Budget

The amount available after paying minimum payments and essential expenses.

Debt Avalanche

A repayment strategy that prioritizes debts with the highest interest rates.

Debt Snowball

A repayment strategy that prioritizes debts with the lowest outstanding balances.

Repayment Schedule

A month-by-month record of interest, payments, and remaining balances.

Debt-Free Date

The estimated date by which all debts will be fully repaid.

Money Value

A financial amount stored internally using integer paise.

EMI

A fixed monthly payment made toward a loan.

2. Overall Description

2.1 Product Perspective

DebtPilot is a standalone desktop application.

The system will have the following logical layers:

1. Presentation layer
2. Application-service layer
3. Domain and calculation layer
4. Persistence layer
5. Reporting and export layer

The financial calculation engine will remain independent of Qt, SQLite, and other external libraries.

This separation will allow the core engine to be reused in future command-line, web, or mobile applications.

2.2 Product Architecture

The application shall follow a layered architecture.

Presentation Layer

Responsible for:

- Displaying screens
- Collecting user input
- Displaying validation messages
- Showing repayment results
- Showing strategy comparisons

Application Layer

Responsible for:

- Coordinating use cases
- Managing financial plans
- Calling repayment strategies
- Saving and loading data
- Preparing data for presentation

Domain Layer

Responsible for:

- Money calculations
- Debt validation
- Interest calculations
- Repayment strategy logic
- Repayment schedule generation
- Financial invariants

Persistence Layer

Responsible for:

- SQLite database access
- Saving financial profiles
- Saving debts
- Saving repayment results
- Loading existing plans

Export Layer

Responsible for:

- CSV generation
 - Repayment schedule export
 - Strategy comparison export
-

2.3 User Classes

Primary User

An individual who wants to:

- Track multiple debts
- Compare repayment strategies
- Understand monthly payment allocation
- Estimate debt-free time
- Reduce total interest

Developer or Reviewer

A technical user who wants to:

- Review the architecture
 - Run tests
 - Inspect financial calculations
 - Extend the system
 - Evaluate the project
-

2.4 Operating Environment

DebtPilot Version 1.0 shall support:

- Windows 10
- Windows 11
- 64-bit systems

Development and build requirements:

- C++20-compatible compiler
- CMake
- Qt 6
- SQLite

- Google Test
- Git
- Visual Studio or compatible IDE

Future support may include Linux and macOS.

2.5 Design Constraints

1. The application shall use C++20.
 2. Money shall be represented internally using integer paise.
 3. Financial calculations shall not use floating-point values for stored balances.
 4. The core calculation engine shall not depend on Qt.
 5. The core calculation engine shall not depend on SQLite.
 6. Financial calculations shall be covered by automated tests.
 7. The application shall store user data locally.
 8. The application shall use deterministic algorithms.
 9. The application shall not claim to provide professional financial advice.
 10. The application shall not execute real financial transactions.
-

2.6 Assumptions

1. Users provide correct financial information.
 2. Interest rates remain constant during a generated repayment plan.
 3. Users make payments according to the generated schedule.
 4. No additional debts are added during a repayment simulation.
 5. No new purchases are added to credit-card balances during a simulation.
 6. Minimum payments remain constant unless the remaining balance is lower.
 7. Interest is applied monthly.
 8. All financial amounts are entered in Indian rupees.
 9. One financial profile represents one user.
 10. The system clock provides the current date.
-

2.7 Dependencies

DebtPilot Version 1.0 depends on:

- Qt 6 for the desktop interface
 - SQLite for local data storage
 - Google Test for automated testing
 - CMake for project configuration
 - Standard Template Library
 - Operating-system file access for CSV export
-

3. Functional Requirements

3.1 Financial Profile Management

FR-001 Create Financial Profile

The system shall allow the user to create a financial profile.

The profile shall contain:

- Profile name
- Monthly net income
- Essential monthly expenses
- Discretionary monthly expenses
- Existing monthly commitments
- Emergency savings contribution
- Preferred plan start date

Acceptance Criteria

- A profile shall not be created without a name.
 - Monthly income shall not be negative.
 - Expense values shall not be negative.
 - The profile shall be saved successfully when all inputs are valid.
 - Validation messages shall be displayed for invalid inputs.
-

FR-002 Edit Financial Profile

The system shall allow the user to update an existing financial profile.

The user shall be able to modify:

- Profile name
- Income
- Expenses
- Monthly commitments
- Savings contribution
- Start date

Acceptance Criteria

- Existing repayment calculations shall be marked outdated after financial details change.
 - The system shall require repayment plans to be recalculated after relevant updates.
-

FR-003 Load Financial Profile

The system shall allow the user to load a previously saved financial profile.

Acceptance Criteria

- All saved profile values shall be restored.
 - All associated debts shall be restored.
 - The most recent repayment results may be displayed if available.
-

FR-004 Delete Financial Profile

The system shall allow the user to delete a financial profile.

Acceptance Criteria

- The user shall receive a confirmation message.
 - The associated debts and saved calculations shall also be deleted.
 - Deletion shall be performed as a database transaction.
-

3.2 Income and Expense Management

FR-005 Record Monthly Income

The system shall allow the user to enter monthly net income.

Validation Rules

- Income shall be zero or greater.
 - Income shall not exceed the configured maximum supported value.
 - Income shall contain at most two decimal places.
-

FR-006 Record Monthly Expenses

The system shall allow the user to enter:

- Essential expenses
- Discretionary expenses
- Fixed commitments
- Emergency savings contribution

Acceptance Criteria

- Each value shall be zero or greater.
 - The system shall calculate total monthly expenses.
 - The system shall display the remaining amount available after expenses.
-

FR-007 Calculate Available Debt Budget

The system shall calculate the amount available for debt repayment.

The debt repayment budget shall be based on:

- Monthly net income
- Essential expenses
- Discretionary expenses
- Monthly commitments
- Savings contribution

The user shall also be allowed to manually define a debt repayment budget.

Acceptance Criteria

- The available debt budget shall not be negative.
 - A warning shall be shown if expenses exceed income.
 - A warning shall be shown when the debt budget is lower than the total minimum payments.
-

3.3 Debt Management

FR-008 Add Debt

The system shall allow the user to add a debt.

Each debt shall contain:

- Unique identifier
- Debt name
- Debt type
- Lender name
- Outstanding balance
- Annual interest rate
- Minimum monthly payment
- Optional EMI
- Optional due day

- Optional notes

Supported debt types shall include:

- Credit card
- Personal loan
- Education loan
- Vehicle loan
- Home loan
- Buy-now-pay-later loan
- Informal loan
- Other

Acceptance Criteria

- Debt name shall be mandatory.
 - Outstanding balance shall be greater than zero.
 - Interest rate shall be zero or greater.
 - Minimum payment shall be zero or greater.
 - Due day, when provided, shall be between 1 and 31.
 - The debt shall receive a unique identifier.
-

FR-009 Edit Debt

The system shall allow the user to edit an existing debt.

Acceptance Criteria

- Changes shall be validated.
 - Existing repayment results shall be marked outdated.
 - Updated debts shall be saved to the database.
-

FR-010 Delete Debt

The system shall allow the user to remove a debt.

Acceptance Criteria

- The system shall request confirmation.
 - Deleted debts shall no longer appear in calculations.
 - Existing repayment plans shall be marked outdated.
-

FR-011 View Debt Summary

The system shall display:

- Number of debts
 - Total outstanding balance
 - Total minimum monthly payment
 - Weighted or average interest summary
 - Highest-interest debt
 - Lowest-balance debt
 - Total monthly debt obligation
-

3.4 Input Validation

FR-012 Validate Financial Values

The system shall validate all financial values before saving or calculating a plan.

Validation Rules

- Currency values shall not contain more than two decimal places.
 - Negative balances shall not be permitted.
 - Negative payments shall not be permitted.
 - Interest rates shall not be negative.
 - Percentage values shall remain within configured limits.
 - Required fields shall not be empty.
 - Invalid values shall not be persisted.
-

FR-013 Detect Insufficient Repayment Budget

The system shall detect when the monthly debt budget is lower than the total required minimum payments.

Acceptance Criteria

The system shall:

- Display the monthly shortage.
 - Prevent generation of a normal repayment plan.
 - Explain that minimum obligations cannot be met.
 - Allow the user to modify income, expenses, debts, or repayment budget.
-

3.5 Interest Calculation

FR-014 Calculate Monthly Interest

The system shall calculate monthly interest for each debt using its annual interest rate.

The default monthly periodic rate shall be derived from the annual interest rate.

Acceptance Criteria

- Zero-interest debts shall accrue no interest.
 - Interest shall be rounded according to the documented rounding policy.
 - Interest shall never create invalid negative values.
 - Monthly interest shall be recorded in the repayment schedule.
 - Calculations shall use deterministic rounding.
-

FR-015 Apply Interest and Payment Order

For each repayment month, the system shall:

1. Record the opening balance.
2. Calculate monthly interest.
3. Add interest to the payable balance.
4. Apply the scheduled payment.
5. Record the closing balance.

The exact order shall remain consistent across all strategies.

FR-016 Handle Final Payments

The system shall ensure that the final payment does not exceed the remaining payable amount.

Acceptance Criteria

- Final balance shall equal zero.
 - Final balance shall not become negative.
 - Unused repayment money shall be transferred to the next eligible debt.
 - The final payment may be lower than the scheduled minimum payment.
-

3.6 Debt Avalanche Strategy

FR-017 Generate Avalanche Repayment Plan

The system shall generate a repayment plan using the Debt Avalanche strategy.

The strategy shall:

1. Pay the minimum amount on every active debt.
2. Allocate the remaining budget to the debt with the highest interest rate.
3. Continue until the targeted debt is fully repaid.
4. Redirect the released amount to the next highest-interest debt.

Tie-Breaking Rules

When two debts have the same interest rate, the system shall prioritize:

1. Lower outstanding balance
2. Earlier creation order
3. Stable unique identifier

Acceptance Criteria

- Every active debt shall receive its required minimum payment where possible.
 - The highest-interest debt shall receive extra repayment.
 - The schedule shall continue until all debts are fully repaid.
 - The plan shall include total months and total interest.
-

3.7 Debt Snowball Strategy

FR-018 Generate Snowball Repayment Plan

The system shall generate a repayment plan using the Debt Snowball strategy.

The strategy shall:

1. Pay the minimum amount on every active debt.
2. Allocate the remaining budget to the debt with the lowest outstanding balance.
3. Continue until the targeted debt is fully repaid.
4. Redirect the released amount to the next lowest-balance debt.

Tie-Breaking Rules

When two debts have the same outstanding balance, the system shall prioritize:

1. Higher interest rate
2. Earlier creation order
3. Stable unique identifier

Acceptance Criteria

- Every active debt shall receive its required minimum payment where possible.
 - The lowest-balance debt shall receive extra repayment.
 - The plan shall continue until all debts are fully repaid.
 - The plan shall include total months and total interest.
-

3.8 Repayment Schedule

FR-019 Generate Month-by-Month Schedule

The system shall generate a detailed monthly repayment schedule.

Each month shall include:

- Month number
 - Calendar month
 - Opening balance for each debt
 - Interest charged
 - Minimum payment
 - Extra payment
 - Total payment
 - Closing balance
 - Debt targeted by the strategy
 - Debts fully repaid during the month
-

FR-020 Display Debt-Free Date

The system shall calculate and display the estimated debt-free date.

The date shall be based on:

- Plan start date
- Number of repayment months
- Monthly repayment schedule

FR-021 Calculate Total Interest

The system shall calculate:

- Total interest for each debt
- Total interest for the complete repayment plan
- Difference in total interest between strategies

FR-022 Calculate Total Amount Repaid

The system shall calculate:

- Principal repaid
- Interest paid
- Total amount paid
- Total months required

FR-023 Detect Non-Progressing Plans

The system shall detect situations where balances do not decrease.

Possible causes include:

- Interest exceeds payment
- Repayment budget is too low
- Minimum payment is zero
- Invalid financial configuration

Acceptance Criteria

- The simulation shall stop safely.
- The user shall receive an explanatory error.
- The application shall not enter an infinite loop.
- The issue shall be recorded in the calculation result.

FR-024 Apply Simulation Limit

The repayment engine shall enforce a maximum simulation duration.

The initial limit shall be configurable, with a default maximum of 1,200 months.

Acceptance Criteria

- The simulation shall stop when the limit is reached.
 - The plan shall be marked incomplete.
 - Remaining balances shall be displayed.
 - A warning shall explain that the debt cannot be repaid under the current configuration.
-

3.9 Strategy Comparison

FR-025 Compare Avalanche and Snowball

The system shall compare both strategies.

The comparison shall display:

- Debt-free date
 - Number of months
 - Total interest
 - Total repayment
 - First targeted debt
 - Repayment order
 - Interest difference
 - Time difference
-

FR-026 Recommend a Strategy Based on Objective

The system may display a non-binding recommendation.

The recommendation rules shall be deterministic.

Examples:

- Recommend Avalanche when minimizing interest is the objective.
- Recommend Snowball when achieving early debt closures is the objective.

The system shall clearly state that the recommendation is informational and not professional financial advice.

3.10 Dashboard

FR-027 Display Dashboard

The dashboard shall display:

- Monthly income
 - Monthly expenses
 - Available repayment budget
 - Total outstanding debt
 - Number of active debts
 - Total minimum payments
 - Highest interest rate
 - Estimated debt-free date
 - Preferred or last-calculated strategy
 - Recent saved plans
-

FR-028 Display Repayment Progress

When a saved repayment plan exists, the system shall display:

- Total debt
- Planned amount repaid
- Remaining balance
- Percentage completed
- Upcoming targeted debt
- Estimated months remaining

Version 1.0 may use manually updated balances rather than automatic bank synchronization.

3.11 Data Persistence

FR-029 Save Financial Data

The system shall save:

- Financial profiles
- Income data
- Expense data
- Debt data
- Calculation settings
- Repayment results

- Repayment schedules
-

FR-030 Load Saved Data

The system shall restore saved data after the application is restarted.

FR-031 Use Local SQLite Database

The application shall use SQLite for local persistence.

Requirements

- Prepared statements shall be used.
 - Database transactions shall be used for multi-step updates.
 - Database schema versioning shall be supported.
 - User input shall not be directly concatenated into SQL statements.
 - Database errors shall be handled safely.
-

FR-032 Handle Database Failure

The system shall handle database errors without crashing.

The system shall:

- Show an understandable error message.
 - Avoid partial writes where transactions are required.
 - Preserve existing valid data.
 - Log technical diagnostic information locally.
-

3.12 Export

FR-033 Export Repayment Schedule to CSV

The system shall allow the user to export a repayment schedule as a CSV file.

The CSV shall contain:

- Strategy
- Month
- Debt name

- Opening balance
 - Interest
 - Minimum payment
 - Extra payment
 - Total payment
 - Closing balance
 - Debt status
-

FR-034 Export Strategy Comparison

The system shall allow the user to export a summary comparison.

The export shall contain:

- Avalanche duration
 - Snowball duration
 - Avalanche interest
 - Snowball interest
 - Total repayment amounts
 - Repayment orders
 - Difference in interest
 - Difference in months
-

3.13 Sample Data

FR-035 Load Demonstration Profile

The system shall provide an optional fictional sample profile.

Sample data shall not contain real user information.

The sample profile shall include:

- Monthly income
 - Expenses
 - At least three debts
 - Different interest rates
 - Different balances
 - Different minimum payments
-

3.14 Application Settings

FR-036 Configure Calculation Settings

The system shall allow configuration of:

- Default plan start date
 - Maximum simulation months
 - Currency display format
 - Rounding policy where applicable
 - Default repayment strategy
-

FR-037 Reset Application Data

The system shall allow the user to clear all locally stored application data.

Acceptance Criteria

- Confirmation shall be required.
 - The action shall be clearly described as irreversible.
 - The database shall be reset safely.
-

4. External Interface Requirements

4.1 User Interface Requirements

The desktop interface shall contain the following screens:

1. Welcome screen
 2. Dashboard
 3. Financial profile screen
 4. Income and expenses screen
 5. Debt management screen
 6. Strategy configuration screen
 7. Repayment-results screen
 8. Strategy-comparison screen
 9. Monthly-schedule screen
 10. Settings screen
-

4.1.1 General Interface Requirements

The user interface shall:

- Use clear labels
 - Use Indian rupee formatting
 - Display validation errors near affected inputs
 - Prevent invalid submissions
 - Support keyboard navigation
 - Provide confirmation dialogs for destructive actions
 - Display loading indicators for longer calculations
 - Maintain consistent layout and typography
-

4.1.2 Currency Display

Currency shall be displayed in Indian rupees.

Examples:

- ₹500
- ₹12,500
- ₹1,25,000
- ₹12,500.50

The displayed value shall be converted from integer paise.

4.1.3 Error Messages

Error messages shall be:

- Clear
- Actionable
- Non-technical where possible
- Associated with the affected field or action

Example:

Your monthly debt budget is ₹4,500 lower than the total minimum payments. Increase the budget or update the debt information before generating a repayment plan.

4.2 Database Interface

The persistence layer shall communicate with SQLite.

The database shall contain logical tables for:

- Profiles
 - Income and expenses
 - Debts
 - Repayment plans
 - Repayment months
 - Debt payments
 - Application settings
 - Schema version
-

4.3 File Interface

The application shall read and write CSV files for export.

The application may load fictional sample data from JSON or internal resources.

5. Data Requirements

5.1 Money Representation

All stored currency values shall use signed 64-bit integer paise.

Examples:

- ₹1.00 shall be stored as 100.
 - ₹100.50 shall be stored as 10,050.
 - ₹1,25,000 shall be stored as 12,500,000.
-

5.2 Percentage Representation

Interest rates shall be stored using a deterministic representation.

Acceptable approaches include:

- Integer basis points
- Fixed-point decimal type
- Controlled decimal representation

Floating-point values shall not be used without documented rounding control.

5.3 Debt Data

Each debt record shall contain:

- Debt ID
 - Profile ID
 - Name
 - Type
 - Lender
 - Balance in paise
 - Annual interest rate
 - Minimum payment in paise
 - EMI amount where applicable
 - Due day
 - Notes
 - Created timestamp
 - Updated timestamp
-

5.4 Repayment Plan Data

Each repayment plan shall contain:

- Plan ID
 - Profile ID
 - Strategy
 - Start date
 - Monthly repayment budget
 - Total principal
 - Total interest
 - Total amount paid
 - Number of months
 - Debt-free date
 - Completion status
 - Calculation timestamp
-

5.5 Monthly Schedule Data

Each monthly schedule entry shall contain:

- Plan ID
 - Month index
 - Calendar month
 - Debt ID
 - Opening balance
 - Interest charged
 - Minimum payment
 - Extra payment
 - Total payment
 - Closing balance
 - Target indicator
 - Paid-off indicator
-

6. Business Rules

BR-001 Currency Rule

Stored financial amounts shall use integer paise.

BR-002 Minimum-Payment Rule

Every active debt shall receive its minimum payment before extra repayment is allocated, provided the available monthly budget is sufficient.

BR-003 Overpayment Rule

A payment shall not exceed the remaining payable amount.

BR-004 Zero-Balance Rule

A debt with a zero balance shall not receive further payments or accrue interest.

BR-005 Avalanche-Priority Rule

The Avalanche strategy shall prioritize the active debt with the highest annual interest rate.

BR-006 Snowball-Priority Rule

The Snowball strategy shall prioritize the active debt with the lowest outstanding balance.

BR-007 Extra-Budget Rule

After minimum payments are allocated, the remaining budget shall be assigned to the strategy's target debt.

BR-008 Rollover Rule

When a debt is fully repaid, its previous minimum payment and allocated extra payment shall become available for future months.

BR-009 Interest Rule

Interest shall be calculated using the documented monthly periodic rate and rounding policy.

BR-010 Final-Payment Rule

The final payment shall equal the remaining payable amount when that amount is less than the planned payment.

BR-011 Insufficient-Budget Rule

A standard repayment plan shall not be generated when the available budget is lower than the required minimum-payment total.

BR-012 Non-Progress Rule

A simulation shall stop when no meaningful debt reduction occurs for a configured number of periods.

BR-013 Plan-Invalidation Rule

Changes to income, expenses, repayment budget, interest rate, balance, or minimum payment shall invalidate previously calculated plans.

7. Non-Functional Requirements

7.1 Performance

NFR-001 Calculation Performance

The system should calculate a repayment plan containing up to:

- 100 debts
- 1,200 repayment months

within two seconds on a typical modern desktop computer.

NFR-002 Startup Performance

The application should display the initial screen within three seconds under normal conditions.

NFR-003 Database Performance

Standard save and load operations should complete within one second for normal user datasets.

7.2 Reliability

NFR-004 Calculation Determinism

Identical inputs and settings shall produce identical repayment results.

NFR-005 Data Integrity

The system shall use transactions for operations that update multiple related database records.

NFR-006 Crash Prevention

Invalid input shall not cause the application to terminate unexpectedly.

NFR-007 Recovery

When possible, the application shall preserve the last successfully saved state following an unexpected failure.

7.3 Security and Privacy

NFR-008 Local Data Storage

Version 1.0 shall store user financial data locally.

NFR-009 No Credential Storage

The application shall not request or store:

- Bank passwords
 - Card PINs
 - CVV numbers
 - Internet-banking credentials
 - One-time passwords
 - Complete payment-card numbers
-

NFR-010 SQL Security

The system shall use prepared statements for database operations involving user data.

NFR-011 Sensitive Logging

Logs shall not contain sensitive financial details unless required for debugging and explicitly enabled.

NFR-012 Sample Data Privacy

All public demonstrations and repository tests shall use fictional financial data.

7.4 Usability

NFR-013 Learnability

A first-time user should be able to create a profile, add debts, and generate a comparison without reading technical documentation.

NFR-014 Input Assistance

The system shall provide:

- Input hints
 - Tooltips
 - Examples
 - Validation messages
 - Explanations of repayment strategies
-

NFR-015 Consistency

Terminology, buttons, currency formats, and validation behaviour shall remain consistent across screens.

7.5 Maintainability

NFR-016 Modular Design

The system shall maintain separation between:

- UI
 - Application services
 - Domain logic
 - Persistence
 - Export functionality
-

NFR-017 Coding Standards

The codebase shall:

- Use C++20
 - Prefer RAII
 - Avoid raw owning pointers
 - Use const-correctness
 - Use meaningful names
 - Enable strict compiler warnings
 - Avoid unnecessary global state
 - Document non-obvious financial rules
-

NFR-018 Automated Tests

The system shall include:

- Unit tests
 - Integration tests
 - Regression tests
 - Financial acceptance tests
-

NFR-019 Test Coverage

Critical financial calculation modules should maintain high behavioural coverage.

Critical modules include:

- Money arithmetic
 - Interest calculation
 - Payment allocation
 - Avalanche strategy
 - Snowball strategy
 - Final-payment handling
 - Insufficient-budget detection
-

7.6 Portability

NFR-020 Build Portability

The project shall use CMake to support reproducible builds.

NFR-021 Platform Independence

Domain logic shall avoid platform-specific dependencies.

7.7 Accessibility

NFR-022 Keyboard Navigation

Primary screens shall support keyboard navigation.

NFR-023 Readability

Text and important financial values shall remain readable under standard display scaling.

NFR-024 Non-Colour Indicators

Status and validation shall not rely exclusively on colour.

7.8 Auditability

NFR-025 Calculation Transparency

The application shall allow users to inspect how each month's payment was allocated.

NFR-026 Calculation Summary

Each repayment result shall include:

- Inputs used
 - Strategy used
 - Calculation timestamp
 - Total interest
 - Duration
 - Repayment order
-

8. Error Handling Requirements

ER-001 Invalid Currency Input

The system shall reject malformed or unsupported currency input.

ER-002 Invalid Interest Rate

The system shall reject negative or unsupported interest rates.

ER-003 Insufficient Budget

The system shall explain when total minimum payments exceed the repayment budget.

ER-004 Database Error

The system shall display a safe error message and preserve valid stored data.

ER-005 Export Error

When CSV export fails, the system shall:

- Display the reason where available.
 - Avoid creating a misleading incomplete file.
 - Allow the user to choose another location.
-

ER-006 Simulation Failure

The system shall stop and report when:

- The plan exceeds the maximum duration.
 - No repayment progress occurs.
 - Arithmetic overflow is detected.
 - Invalid internal state is encountered.
-

9. Testing Requirements

9.1 Unit Testing

Unit tests shall cover:

- Money construction
 - Money addition
 - Money subtraction
 - Money comparison
 - Currency formatting
 - Interest calculation
 - Payment capping
 - Avalanche prioritization
 - Snowball prioritization
 - Tie-breaking rules
 - Final payments
 - Zero-interest debts
 - Zero-balance debts
 - Invalid input
 - Insufficient budget
 - Non-progressing plans
-

9.2 Integration Testing

Integration tests shall cover:

- Saving and loading profiles
 - Saving and loading debts
 - Saving repayment plans
 - Database transaction behaviour
 - Temporary database creation
 - CSV export
 - Application-service and repository integration
-

9.3 Acceptance Testing

Acceptance tests shall include manually verified financial examples.

At least the following scenarios shall be included:

1. Single zero-interest debt
 2. Single interest-bearing debt
 3. Multiple debts with different interest rates
 4. Equal-interest debts
 5. Equal-balance debts
 6. Final partial payment
 7. Debt budget exactly equal to minimum payments
 8. Debt budget below minimum payments
 9. Payment lower than monthly interest
 10. All debts already cleared
-

10. User Stories

US-001

As a user, I want to enter all my debts so that I can view my total outstanding balance.

US-002

As a user, I want to enter income and expenses so that the application can calculate how much I can use for repayment.

US-003

As a user, I want to compare Avalanche and Snowball strategies so that I can choose a suitable repayment method.

US-004

As a user, I want to see monthly payments so that I know how much to pay toward each debt.

US-005

As a user, I want to see my estimated debt-free date so that I can understand how long repayment may take.

US-006

As a user, I want to know the total interest under each strategy so that I can evaluate the financial cost.

US-007

As a user, I want my data to remain available after restarting the application.

US-008

As a user, I want to export my repayment schedule so that I can review it outside the application.

US-009

As a user, I want clear validation messages so that I can correct incorrect entries.

US-010

As a user, I want to inspect how each payment was calculated so that I can trust the result.

11. Version 1.0 Acceptance Criteria

DebtPilot Version 1.0 shall be considered complete when:

1. A user can create and save a financial profile.
2. A user can add, edit, and remove debts.
3. The application validates all required financial inputs.
4. The application calculates the monthly debt repayment budget.
5. The application generates an Avalanche repayment plan.
6. The application generates a Snowball repayment plan.
7. Both plans include month-by-month payment schedules.
8. Both plans include total interest and debt-free dates.
9. The application compares both strategies.
10. Final payments do not create negative balances.
11. Insufficient repayment budgets are detected.
12. Non-progressing simulations terminate safely.
13. Financial data is stored locally using SQLite.
14. Saved data can be restored after restarting the application.

15. Repayment schedules can be exported to CSV.
 16. Critical financial rules are covered by automated tests.
 17. At least five manually verified calculation scenarios pass.
 18. The application runs on Windows 10 and Windows 11.
 19. The core engine remains independent of Qt and SQLite.
 20. The GitHub repository contains build instructions, screenshots, architecture documentation, tests, and sample data.
-

12. Future Enhancements

Possible features for later versions include:

Version 1.1

- Budget categories
- Recurring-expense management
- Additional repayment strategies
- PDF report export
- Charts and repayment visualizations
- Actual-payment tracking
- Plan-versus-actual comparison

Version 2.0

- Bank-statement import
- Transaction categorization
- AI-generated explanations
- Scenario simulations
- Emergency-fund recommendations
- Interest-rate-change simulations
- Promotional credit-card interest periods
- Variable minimum-payment models

Version 3.0

- Cloud synchronization
 - Mobile application
 - Multi-device access
 - Household financial profiles
 - Secure account authentication
 - Optional financial-data encryption
 - External financial-data integrations
-

13. Risks

Risk 1: Incorrect Financial Calculations

Incorrect calculations may provide misleading repayment projections.

Mitigation

- Use integer paise.
 - Maintain deterministic rounding.
 - Create manually verified acceptance tests.
 - Add regression tests for every calculation defect.
-

Risk 2: Overengineering

Complex architecture may delay Version 1.0.

Mitigation

- Limit the initial feature set.
 - Avoid unnecessary abstractions.
 - Introduce interfaces only where multiple implementations are required.
-

Risk 3: AI-Generated Code Errors

AI-assisted implementation may introduce incorrect assumptions or unnecessary complexity.

Mitigation

- Review every generated change.
 - Require automated tests.
 - Verify formulas independently.
 - Avoid merging code that cannot be explained.
-

Risk 4: Sensitive Financial Data Exposure

Real financial information may accidentally be committed to the repository.

Mitigation

- Use fictional sample data.

- Add database and personal-data files to `.gitignore`.
 - Do not log sensitive values by default.
 - Review commits before pushing.
-

Risk 5: Infinite Repayment Simulations

Certain combinations of interest and payment values may prevent debts from decreasing.

Mitigation

- Detect non-progressing balances.
 - Enforce a simulation limit.
 - Validate repayment budgets.
 - Stop and report incomplete plans.
-

14. Disclaimer

DebtPilot is an educational and informational financial-planning application.

It does not provide:

- Professional financial advice
- Credit counselling
- Investment advice
- Tax advice
- Loan approval advice
- Legal advice
- Guarantees of financial outcomes

Users remain responsible for verifying repayment amounts, lender requirements, fees, interest calculations, and actual payment obligations.

15. Approval

The requirements in this document define the planned scope of DebtPilot Version 1.0.

Changes to critical financial rules shall require:

1. Documentation
2. Updated acceptance criteria
3. Updated automated tests

4. Regression verification
5. Version-control review

Document Status: Approved for Initial Development

Target Release: DebtPilot Version 1.0